

퀄리저널 개발 파일 사전

루트 프로젝트 (기존 스크립트 기반 시스템)

QualiJournal의 초기 버전은 Windows 환경에서 PowerShell 스크립트와 파이썬 모듈로 구성되어 수집부터 발행까지 자동화된 **뉴스 큐레이션 파이프라인**을 구현했습니다. JSON 파일(`selected_keyword_articles.json`, `selected_articles.json`)을 통해 기사 목록과 편집 결과를 저장/관리하며, 매일 수동 또는 예약된 방식으로 스크립트를 실행해 **일일 보고서**를 생성했습니다. 아래는 루트(zip 압축본) 내 주요 파일들의 사전입니다.

주요 파이썬 모듈 (상위 디렉터리)

파일명	경로	주요 기능
<code>engine_core.py</code>	<code>/engine_core.py</code>	뉴스 수집 및 품질검증(품질 게이트, 팩트체크)와 기사 점수 평가, 선정/발행의 핵심 로직을 담당하는 모듈입니다. 공식 소스(뉴스)와 커뮤니티 소스를 수집하여 메타데이터 정제 및 QG/FC (품질 게이트/팩트체크) 처리를 수행하고, 각 기사에 점수를 부여합니다. 또한 번역 API를 활용한 요약/번역 기능과 키워드별 기사 히스토리 관리 등을 포함한 뉴스 파이프라인의 핵심 구현체 입니다. (예: 커뮤니티 글의 경우 AI 토픽 제외, 업보트/댓글수/최근성 등의 기준 적용 등)
<code>orchestrator.py</code>	<code>/orchestrator.py</code>	QualiJournal 전체 수집-발행 프로세스를 오케스트레이션 하는 상위 모듈입니다. <code>engine_core</code> 등의 모듈을 호출하여 공식 뉴스와 커뮤니티 자료를 각각 수집 처리하고, 편집자 승인 여부에 따라 최종 선정된 기사들을 Markdown/HTML/JSON 일일보고서로 발행 합니다. 공식/커뮤니티 소스를 분리 운영하며, 수집된 기사들의 승인/편집 상태를 반영해 아카이브 파일 생성 , 로그 기록(<code>logs/server.log</code> 사용) 등을 수행합니다. (이 모듈을 통해 전체 파이프라인을 스케줄링하며, PowerShell 스크립트에서 이 모듈을 실행해 일일 작업을 진행합니다.)
<code>common_utils.py</code>	<code>/common_utils.py</code>	여러 컴포넌트에서 공통으로 사용하는 유틸리티 함수들을 모은 모듈 입니다. JSON 및 경로 처리, 환경 설정 로드, 문자열 처리 등 중복되는 헬퍼 기능들을 중앙화 하여 <code>orchestrator</code> , <code>engine_core</code> 등에서 가져다 씁니다. 예를 들어 시간/날짜 포맷 변환, 뉴스 도메인 관련 규칙 처리, 또는 config 파싱 등의 기능을 제공합니다.
<code>config_schema.py</code>	<code>/config_schema.py</code>	설정 스키마 정의 모듈로 , <code>config.json</code> 의 구조와 기본값을 정의하거나 검증하는 역할을 합니다. QualiJournal 설정 파일의 필드 (예: 임계값, 옵션 토글 등)가 유효한지 확인하고, 누락된 값에 기본값을 적용하는 등 설정 관리의 일관성 유지 를 지원합니다.

파일명	경로	주요 기능
qj_paths.py	/qj_paths.py	프로젝트 디렉터리 경로 관리 모듈입니다. 데이터/아카이브 폴더 등 자주 사용하는 경로를 정의하고, <code>qj_paths.rel()</code> 등의 함수를 통해 코드 전반에서 상대 경로를 편리하게 참조 할 수 있게 해줍니다. 예를 들어 <code>data/</code> 나 <code>archive/</code> 디렉터리의 경로를 쉽게 얻어 파일 입출력에 활용합니다.
logging_setup.py	/logging_setup.py	로그 설정 모듈로 , 파일 및 콘솔 출력 로거를 초기화합니다. 로깅 포맷, 레벨 등을 지정하고, 주요 모듈에서 이를 import하여 일관된 로깅 을 수행합니다. (예: <code>server.log</code> 파일에 debug/info 로그를 남겨 추후 분석 가능)

구성/환경 설정 파일

파일명	경로	주요 기능
config.json	/config.json	QualiJournal의 핵심 설정 파일로 , 뉴스 선별 임계값, 기능 토글, 소스 목록 경로 등을 저장합니다. 예를 들어 커뮤니티 게시물 최소 업보트/댓글 수, 점수 임계값과 품질게이트 동적 임계값 설정, 소스 리스트 경로 등이 포함됩니다 ¹ . 운영 중 이 값을 조정하여 필터링 강도를 제어할 수 있습니다. (관련 이슈: 임계값이 정적으로 설정되어 키워드 특성에 따라 엄격도 차이가 발생할 수 있어, 상황별 동적 조정이나 ML기반 개선 필요성이 제기되었습니다.)
.env	/.env	환경변수 설정 파일로 , ADMIN_TOKEN, API_KEY, DB_URL 등 비밀키 및 환경별 설정값을 담고 있습니다. 개발/운영 환경에서 공통으로 참조되며, 서버 실행 전 이 값을 로드해 보안 토큰 및 외부 API 키 등을 설정합니다. (주의: Git에 커밋되지 않도록 .gitignore 처리되며, 실제 배포 시에는 Secret Manager 등을 통해 주입)
.env.example	/.env.example	환경변수 예시 파일로 , <code>.env</code> 에 넣어야 할 키들의 포맷을 보여줍니다. 예를 들어 <code>ADMIN_TOKEN="your-admin-token"</code> 등의 형식으로 값 없이 키만 제공하여, 새로운 개발자가 환경변수를 쉽게 구성하도록 돕습니다.
.github/workflows/ci.yml	/.github/workflows/ci.yml	GitHub Actions CI/CD 파이프라인 정의 파일입니다. 코드 푸시 시 자동으로 테스트 및 빌드, GCP Cloud Run 배포 등을 수행하는 워크플로우가 기술되어 있습니다. (예: main 브랜치 커밋 → 컨테이너 빌드 및 Cloud Run 자동 배포, WIF로 GCP 연동 등) 이를 통해 코드 변경이 자동으로 배포 되도록 합니다.

PowerShell 스크립트 (/scripts 디렉터리)

QualiJournal의 기존 실행 흐름은 Windows PowerShell 스크립트로 자동화되었습니다. 스크립트들은 정해진 일정에 **일괄 작업**을 수행하거나 운영 편의를 위해 작성되었으며, 일부는 Windows 환경에 종속적이어서 경로 인코딩 문제 및 유지보수에 어려움이 있었습니다. (실제로 PowerShell 기반 자동화는 f-string 해석 오류, 경로 인코딩 문제 등으로 유지보수 부담이 컸던 이슈가 있습니다.) 각 스크립트의 기능은 다음과 같습니다:

파일명	경로	주요 기능	호출 관계/종속성
run_quali_today.ps1	/scripts/ run_quali_today.ps1	일일 뉴스 수집 및 발행 파이프라인을 실행 하는 메인 스크립트입니다. 하루 한 번 전체 프로세스를 돌려 해당일의 키워드 뉴스 특별호를 생성합니다. 내부적으로 Python 환경을 세팅하고 <code>orchestrator.py</code> 등을 호출하여 당일 기사 수집→품질평가→발행 까지 수행합니다.	<code>orchestrator.py</code> 모듈 호출, Python 실행 환경 필요
run_daily_manual.ps1	/scripts/ run_daily_manual.ps1	수동 일일 실행 을 위한 스크립트로, 자동 스케줄러 대신 운영자가 원할 때 당일 파이프라인을 실행할 수 있게 합니다. 주로 개발/테스트나 스케줄러 장애 시 수동 백업 절차로 사용됩니다.	<code>run_quali_today.ps1</code> 또는 유사 모듈을 내부 호출
run_weekly_report.ps1	/scripts/ run_weekly_report.ps1	주간 보고서 생성 을 위한 스크립트입니다. 일주일 간의 누적 결과나 로그를 정리해 별도의 주간 리포트를 만들 때 사용합니다. (예: 주간 통계 요약)	누적 데이터 필요 (일일 보고서 아카이브 등)
run_log_report_weekly.ps1	/scripts/ run_log_report_weekly.ps1	주간 로그 리포트 를 생성하는 스크립트입니다. 서버/수집 로그를 모아 일주일 단위로 분석한 결과를 보고서 형태로 정리합니다.	<code>logs/server.log</code> 등 로그 파일 입력
run_log_report_monthly.ps1	/scripts/ run_log_report_monthly.ps1	월간 로그 리포트 를 생성하는 스크립트로, 한 달치 로그 데이터를 분석/요약합니다. 주간 리포트와 유사하나 기간만 월간으로 확대합니다.	로그 파일 입력

파일명	경로	주요 기능	호출 관계/종속성
run_backup_gcs.ps1	/scripts/ run_backup_gcs.ps1	백업 스크립트로 , 생성된 JSON/MD 결과물들을 주기적으로 GCP의 Cloud Storage 등에 업로드하여 백업합니다. (예: archive/ 폴더의 파일들을 GCS 버킷에 저장)	gsutil/Cloud SDK 명령 호출
deploy_ci.ps1	/scripts/deploy_ci.ps1	CI 배포 스크립트로 , GitHub Actions 등 CI 환경에서 실행하여 최신 코드를 빌드/배포하는 자동화 절차를 PowerShell로 작성한 것입니다. (예: Docker 이미지 빌드, Artifact Registry 푸시 등)	Cloud SDK, Docker 등
smoke.ps1	/scripts/smoke.ps1	Smoke 테스트 스크립트로 , 배포 후 주요 API 엔드포인트 (/health, /api/status 등)에 간단한 요청을 보내 정상 응답 (200 OK) 을 확인합니다. 배포 검증 및 모니터링 용도로 사용됩니다.	curl 등 HTTP 요청 사용
start_win.ps1	/scripts/start_win.ps1	로컬 개발 서버 기동 스크립트(Windows)입니다 . Windows 환경에서 필요한 설정 (.env 로드 등)을 수행한 뒤 Uvicorn 또는 Flask 개발 서버를 실행해 로컬에서 테스트할 때 사용합니다.	Python 실행, 환경변수 로드 (.env)

도구 Python 스크립트 (/tools 디렉터리)

tools/ 폴더에는 수집된 데이터의 **보정/추가 처리 및 발행 보조**를 위한 다양한 유틸리티 스크립트들이 모여 있습니다. 2 3. 이들은 필요에 따라 수동으로 실행하여 데이터 품질을 높이거나 긴급 상황에 대응하기 위한 도구들입니다. 주요 스크립트는 다음과 같습니다:

파일명	경로	주요 기능
make_daily_report.py	/tools/make_daily_report.py	일일 보고서 만들기 현재까지 승인된 (selected_articles) 일 특별호 보고서입니다. 4. 생성된 reports/ 폴더에 DD_report.m... 코멘트와 모든 승인 API (/api/reports) 스크립트를 실행합니다. (5.)
enrich_cards.py	/tools/enrich_cards.py	뉴스 카드 요약 및 전체 기사 목록 (selected_keywords)을 읽어 각 기사에 번역을 수행하고, summary, title, 4. (요약에는)에는 Papago API를 호출합니다. 리포트는 편집자가 생성할 때 사용됨
rebuild_kw_from_today.py	/tools/rebuild_kw_from_today.py	금일 수집된 원본으로 재작성하는 부터 수집된 raw selected_keywords 형식으로 일관된입니다. 6. (예: 다 통합) 이를 통해 데이터를 사용합니
augment_from_official_pool.py	/tools/augment_from_official_pool.py	공식 뉴스풀 보강 가 기준치(예: 15)에서 추가 기사질 점수가 높은 순하여 특별호 최소

파일명	경로	주요 기능
force_approve_top20.py	/tools/force_approve_top20.py	상위 20개 기사의 수가 가장 높은 20개 기사를 approved=tr 에 승인 처리합니 는 대신 임계치 0 사용됩니다. (동명 성 있음)
force_approve_top40.py	/tools/force_approve_top40.py	상위 40개 기사 별한 상황(예: 키 때) 점수 상위 40 동작은 force_ 사하나 대상 개수
force_set_selected_and_approved_top20.py	/tools/ force_set_selected_and_approved_top20.py	상위 20개 기사를 환하는 스크립트 selected_ke 서 상위 20개를 글 approved=tr 대상이 되도록 만 을 단축)
fix_kw_approve_min15.py	/tools/fix_kw_approve_min15.py	키워드별 최소 승 니다. 승인된 기사 추가 승인하거나 를 확보합니다. augment_fro 함께 사용되어 발
merge_and_approve_top30.py	/tools/merge_and_approve_top30.py	상위 30개 기사 여러 소스에서 상 스트로 병합하고 식 + 커뮤니티 통
normalize_and_approve_top20.py	/tools/normalize_and_approve_top20.py	상위 20개 기사 니다. 점수 분포를 후 상위 20개를 글 점수 편차를 보정 려는 목적입니다.

파일명	경로	주요 기능
promote_keyword_to_selection.py	/tools/promote_keyword_to_selection.py	키워드 기사 선정 정하면 관련 기사 selected=tr 자가 키워드 특별 드의 모든 기사를 행 단계로 넘깁니
quick_publish_keyword.py	/tools/quick_publish_keyword.py	키워드 뉴스 긴급 키워드에 대한 수 에 수행합니다. (S 정으로 실행) 이를 스를 즉시 발행할
ready_gate.py	/tools/ready_gate.py	품질 게이트 준비 재 수집된 기사들 족하는지 미리 점 줍니다. (예: 승인 점수 분포가 적절
rebuild_from_archive_force15.py	/tools/rebuild_from_archive_force15.py	아카이브 데이터 archive/ 에 요 데이터를 재구 최소 15개로 맞추 료의 포맷 업데이트
repair_selection_files.py	/tools/repair_selection_files.py	선정 JSON 파일 selected_ke selected_ar 치나 잘못된 필드 니다. (예: 잘못된
sync_selected_for_publish.py	/tools/sync_selected_for_publish.py	발행 대상 동기화 selected_ke 상의 approve selected_ar 화해 줍니다. 편집 어긋날 수 있는 상

파일명	경로	주요 기능
url_check.py	/tools/url_check.py	소스 URL 상태 점검 나 RSS 피드 URL 코드 등을 검사하 에 발견합니다. (구 요)
build_assets_hash.py	/tools/build_assets_hash.py	프론트엔드 에셋 리자 UI의 JS/CS 변경되도록 함으 효율을 유도합니 고유 hash를 추가
_d4_utils.py (보조)	/tools/_d4_utils.py	내부 보조 유틸 도 으로 사용하는 함 (예: JSON 직렬화 파일명 앞의 <code>_</code> 를
init.py (패키지)	/tools/__init__.py	tools 디렉터 기 위한 파일로, <code>__</code>

데이터 및 피드 파일

QualiJournal 시스템은 JSON 파일을 통해 수집 데이터와 편집 결과를 관리합니다. 또한 별도의 `feeds/` 폴더에 뉴
스 소스와 편집 규칙에 관한 설정이 있습니다.

파일명	경로	주요 내용
selected_keyword_articles.json	/data/ selected_keyword_articles.json (또는 archive/ selected_keyword_articles.json)	당일 수집된 전체 기사 목록을 담은 JSON 파 일입니다. 키워드 기준으로 수집된 모든 기사 들의 제목, 요약, 출처, 날짜, 점수, 승인 여부 등이 저장됩니다 ⁸ . 편집자는 이 파일을 열 어 각 기사에 approved 플래그와 editor_note (편집 코멘트)를 추가합니다 ⁹ .

파일명	경로	주요 내용
selected_articles.json	<code>/data/selected_articles.json</code>	최종 발행 대상 기사 목록 JSON 파일입니다. 위 <code>selected_keyword_articles.json</code> 중 편집자가 승인한 기사들만 모아 저장한 것으로, 일일 보고서 생성 시 사용되는 확정본 입니다 ¹⁰ . (일반적으로 하루 15건의 기사가 이 파일에 담깁니다.)
community_sources.json	<code>/feeds/community_sources.json</code>	커뮤니티 뉴스 소스 목록 JSON입니다. Reddit 등 커뮤니티 출처들의 RSS/API URL, 유형, 가중치 등이 포함되어 있습니다. 커뮤니티 자료 수집 시 이 목록을 참조하여 각 소스에서 최신 게시물을 가져옵니다. (<code>feeds/backup/community_sources.json</code> 은 예전 버전 백업)
official_sources.json	<code>/feeds/official_sources.json</code>	공식 뉴스 소스 목록 JSON입니다. 주요 공식 뉴스 사이트나 블로그의 피드 URL과 메타정보를 담고 있으며, 일일 뉴스 수집 시 활용됩니다. (예: 산업표준 관련 공식 발표 RSS 등)
editor_rules.json	<code>/feeds/editor_rules.json</code>	편집 점수 규칙 JSON입니다. 도메인별 또는 키워드별 가중치, 최신성 가중치, 이슈 중요도 등 기사 가치 평가 규칙 이 정의되어 있습니다. QualiJournal의 평가 계층(Evaluator)에서 이 파일의 기준을 참고하여 각 기사에 점수를 부여합니다 ¹¹ .

※ **Archive 폴더**(`archive/`): 발행된 결과물들을 날짜별로 보존하는 디렉터리입니다. `reports/` 하위에 **생성된 일일 보고서**의 Markdown/HTML 파일이 저장되고, `enriched/` 하위에 요약/번역이 반영된 중간 결과 Markdown 등이 기록됩니다 ¹². 또한 `archive/backup/`에는 각 날짜별 발행본의 JSON/MD/HTML 스냅샷이 (키워드별 파일명으로) 남겨져 **히스토리 확인 및 재분석**이 가능하게 합니다 ¹³ **【40+】**.

QJ_QGFC_DB_Patchset (품질게이트/DB 패치셋)

`QJ_QGFC_DB_Patchset/` 디렉터리는 QualiJournal에 **데이터베이스 연동 및 품질게이트(QG)/팩트체크-점수 평가 개선**을 도입하기 위해 준비된 패치 코드 모음입니다. 기존 JSON 기반 저장을 DB로 마이그레이션하고, 품질 평가 로직을 정교화하기 위한 실험적 코드가 포함되어 있습니다.

파일명	경로	주요 기능
qgfc.py	<code>/QJ_QGFC_DB_Patchset/app/qgfc.py</code>	QG(품질게이트) & FC(팩트체크) 로직 구현 모듈입니다. 기사 본문 및 메타데이터를 입력 받아 자동 품질 검증 및 사실 확인 절차를 수행하고 점수를 계산하는 함수들이 포함되어 있습니다. (예: 저품질 기사 필터링, 중복 검출, 사실출처 교차확인 등 알고리즘 구현)

파일명	경로	주요 기능
import_from_json.py	/QJ_QGFC_DB_Patchset/app/ import_from_json.py	기존 JSON 데이터를 DB로 적재하거나 새로운 구조로 변환하는 모듈입니다. selected_keyword_articles.json 등의 파일을 읽어 DB 스키마에 맞게 매핑/삽입하거나, DB 없이도 메모리 구조체로 import하여 활용할 수 있도록 하는 함수들을 제공합니다.
qj_db.py	/QJ_QGFC_DB_Patchset/app/ qj_db.py	QualiJournal 전용 DB 인터페이스 모듈입니다. PostgreSQL 등 관계형 DB에 대한 연결 설정과 쿼리 함수, ORM 모델 등을 정의하여 기사, 키워드, 로그 등을 데이터베이스로 저장/조회할 수 있게 합니다. (DB_URL 설정에 따라 로컬 또는 Cloud SQL에 연결)
server_quali_patch.py	/QJ_QGFC_DB_Patchset/ patches/ server_quali_patch.py	서버 패치 스크립트: 기존 server_quali.py에 DB 연동 및 개선된 로직(QG/FC 적용 등)을 반영하기 위한 수정 사항을 담고 있습니다. FastAPI 경로 수정, 의존성 주입 변경 등이 이 패치에 기술되어 있습니다. (예: API 호출 시 JSON 대신 DB 조회로 변경하는 코드)
orchestrator_patch.py	/QJ_QGFC_DB_Patchset/ patches/ orchestrator_patch.py	오케스트레이터 패치 스크립트: 기존 orchestrator.py의 일부 로직을 교체/추가하기 위한 패치 내용입니다. DB 도입에 따른 데이터 입출력 경로 변경, 품질평가 부분에서 qgfc.py 함수 호출 등 파이프라인 개선점이 주석과 함께 포함되어 있습니다.
0001_init.sql >0002_indexes.sql	/QJ_QGFC_DB_Patchset/db/ migrations/0001_init.sql /QJ_QGFC_DB_Patchset/ db/migrations/ 0002_indexes.sql	데이터베이스 마이그레이션 SQL 스크립트들입니다. 0001_init.sql은 QualiJournal DB의 초기 스키마 (테이블 생성 등)를 정의하고, 0002_indexes.sql은 성능 향상을 위한 인덱스를 추가합니다. (예: 기사 테이블에 index 설정)
schema.sql	/QJ_QGFC_DB_Patchset/db/ schema.sql	DB 스키마 정의 SQL로, 모든 테이블 구조를 한 파일에 정리해 보여줍니다. (마이그레이션 SQL을 합친 개념도) 이를 통해 DB 구조 (기사, 키워드, 로그 등의 테이블 및 관계)를 한눈에 파악할 수 있습니다.
requirements.additions.txt	/QJ_QGFC_DB_Patchset/ patches/ requirements.additions.txt	추가 의존성 목록으로, 패치셋 적용 시 필요한 파이썬 패키지들을 명시합니다. (예: SQLAlchemy, psycopg2 등 DB 연동 및 새로운 기능에 필요한 라이브러리)

(참고: 위 패치셋은 최종 Admin 시스템에 일부 반영되었습니다. Admin 코드의 app/ 모듈들이 이 패치셋의 개선 내용을 기반으로 작성되어, JSON+파일 기반 시스템에서 DB지원 웹서비스로 발전하게 되었습니다.)

관리자 시스템 (Admin 프로젝트)

QualiJournal 관리자(Admin) 웹 시스템은 편집자가 브라우저 상에서 뉴스를 관리하고 발행할 수 있도록 구축된 **FastAPI 백엔드 + 단일 페이지 프론트엔드** 애플리케이션입니다. 2025년 10월 기준 최신 개발 버전으로, 앞서 루트 프로젝트의 기능들을 웹 UI화하고 백엔드 API로 제공함으로써 **실시간 대시보드**와 **원클릭 발행**을 구현했습니다 ¹⁴. 아래는 admin(zip 압축본) 내 각 파일의 설명입니다.

핵심 백엔드 파일 (FastAPI 서버 및 유틸)

파일명	경로	주요 기능	연관 모듈/파일 및 종속성	관련 API 엔드포인트
server_quali.py	/server_quali.py	FastAPI 기반 백엔드 서버 메인 파일 로, 관리자 (Admin) API 엔드포인트들을 모두 정의하는 엔트리포인트입니다. 서버 기동 시 FastAPI(app) 앱 객체를 생성하고 index.html 정적 서빙 설정, CORS 및 토큰 인증 미들웨어 적용 등을 수행합니다 ¹⁴ . 주요 기능으로 일일 보고서 생성, 기사 요약/번역, 결과물 내보내기(MD/CSV), 상태 조회 등을 담당하는 다양한 API 라우트를 정의하며, 내부적으로 JSON 데이터 및 각종 스크립트를 호출하여 작업을 처리합니다 ¹⁴ . 예를 들어 /api/report 호출 시 현재 승인 기사들을 취합해 보고서를 만들고, /api/enrich/selection 호출 시 기사 요약을 수행합니다. 서버 시작 시 config.json (또는 Env 설정)을 로드하여 임계값 등을 초기화하며, ADMIN_TOKEN 검증 로직과 CORS 설정도 포함됩니다 ¹⁴ .	auth_utils.py (토큰 검증), app/qj_db.py (데이터 접근), app/qgfc.py (요약/평가) 등 모듈을 import. 또한 index.html 및 정적 /assets 파일 제공	다수의 API 엔드포인트를 직접 구현 (FastAPI 데코레이터 사용): 예) /api/status (상태 점검), /api/report (POST; 일일 보고서 생성), /api/enrich/keyword & /api/enrich/selection (POST; 요약/번역 실행), /api/export/md & /api/export/csv (GET; 보고서 MD 또는 CSV 다운로드), /api/selection (GET/POST; 기사 선정 현황 조회/업데이트), /api/items 관련 (기사 목록 및 발행), /api/logs (로그 조회) 등 다양한 Admin API 엔드포인트들을 관장합니다 【32 +】 .

파일명	경로	주요 기능	연관 모듈/파일 및 종속성	관련 API 엔드포인트
auth_utils.py	/auth_utils.py	인증/인가 관련 유틸리티 모듈로, Admin 토큰 검증 등의 함수를 제공합니다. HTTP 요청 헤더의 X-Admin-Token 값을 확인하여 유효한 토큰 (환경변수의 ADMIN_TOKEN과 일치)인지 검증하고, FastAPI Dependency로 활용되어 모든 보호된 API 엔드포인트에 인증을 적용합니다. 이 외에 토큰 관련 부가 기능(예: 만료 시간 검토나 에러 메시지 정의)이 구현되어 있습니다.	환경변수 (ADMIN_TOKEN) 값을 참조하며, FastAPI Depends 로 server_quali.py 내부 엔드포인트에 주입되어 사용됩니다.	(직접 엔드포인트는 없음) - server_quali.py 내 각 API 라우트의 dependency로 활용)

파일명	경로	주요 기능	연관 모듈/파일 및 종속성	관련 API 엔드포인트
db.py	/db.py	데이터베이스 연결 및 설정 관리 모듈입니다. PostgreSQL 등의 DB와 연결하기 위한 SQLAlchemy 엔진 초기화, Cloud SQL Connector 사용 설정 【54†】, 및 DB 접속 함수를 제공합니다. DB_URL 환경변수를 읽어 로컬 또는 Cloud SQL에 연결 설정을 수행하며, 필요 시 Google Cloud의 IAM 인증을 사용한 보안 접속도 지원합니다. 또한 DB 세션 관리, 테이블 자동 생성 (필요시) 등을 처리합니다. (이 모듈을 통해 Admin 시스템이 JSON 파일 대신 DB를 사용할 수 있게 되며, DB_URL 미설정 시에는 파일 모드로 동작합니다.)	SQLAlchemy, pg8000 드라이버 등 외부 패키지에 의존 【54†】. app/qj_db.py에서 이 모듈을 import하여 사용.	(직접적인 API 엔드포인트는 없으나, DB 상태 점검 용도로 /api/db/ping, /api/db/mode 등을 server_quali.py에서 구현하여 이 모듈 통해 DB 연결 여부를 확인)
init.py	/__init__.py	패키지 초기화 파일로 특별한 로직은 없습니다. (app/, admin/ 디렉토리 외에 루트 경로에 이 파일이 있어, 전체를 하나의 패키지처럼 인식시키는 용도입니다.) 내용은 비어 있습니다.	-	-

품질게이트/DB 연동 모듈 (/app 패키지)

Admin 시스템에서 **기사 데이터 처리**와 **DB 연동**에 관련된 주요 로직은 app/ 패키지 내 모듈들로 구성되어 있습니다. 이는 이전 패치셋(QGFC_DB)의 개선 사항을 반영하여, JSON 기반 데이터를 다루던 코드를 재구성한 것입니다.

파일명	경로	주요 기능	호출 관계/종속성
qgfc.py	/app/qgfc.py	품질 게이트(QG) 및 팩트체크(FC) 핵심 로직 모듈입니다. 기사들의 점수를 계산하고 필터링하는 함수, 예를 들어 낮은 품질 기사 배제, 중복 검출, 사실확인 등의 알고리즘을 구현합니다. FastAPI 엔드포인트에서 요약/번역 실행 전후로 호출되어, 임계값 설정(<code>config.json</code> 또는 DB값)을 기준으로 기사 통과 여부를 판정합니다.	<code>app/qj_db.py</code> 또는 JSON 데이터로부터 기사 목록을 입력받아 처리. <code>server_quali.py</code> 의 <code>/api/enrich/*</code> 경로 등에서 이 모듈함 사용.
import_from_json.py	/app/ import_from_json.py	JSON 데이터 가져오기 및 변환 모듈입니다. 기존 <code>selected_keyword_articles.json</code> 이나 <code>selected_articles.json</code> 파일을 읽어와 파이썬 객체나 DB에 삽입할 수 있는 형태로 파싱합니다. Admin 서버 초기화 시 기존 JSON 파일이 있으면 읽어서 DB를 채우거나, 필요 시 편집 UI에서 JSON을 가져와 화면에 보여줄 때도 사용됩니다.	Python 표준 <code>json</code> 모듈 사용. <code>server_quali.py</code> 일부 루틴(예: 서버 시작 시 데이터 로드)에 호출되거나, <code>/api/dev/seed/keyword</code> 같은 개발용 엔드포인트에서 용될 수 있음.
qj_db.py	/app/qj_db.py	QualiJournal 데이터 접근 레이어 모듈입니다. DB를 사용할 경우 기사, 키워드, 선택/승인 데이터를 조회/갱신하는 함수들을 제공하고, DB가 없을 경우에는 내부적으로 JSON 파일을 대신 사용하도록 추상화되어 있습니다. 예를 들어 승인된 기사 목록 가져오기, 기사 승인 상태 업데이트 등의 함수가 구현되어 있어 백엔드 엔드포인트에서 호출됩니다. 또한 DB 스키마에 맞춘 ORM 모델이나 쿼리문이 포함되어 있어, DB 사용 시에는 이 모듈만 수정하면 나머지 코드가 영향받지 않게 설계되었습니다.	<code>db.py</code> 로부터 생성된 DB 엔진/세션을 사용. <code>server_quali.py</code> 다수 API에서 이 모듈의 함수를 호출하여 DB 또는 JSON으로부터 데이터를 읽고 씁니다.

관리자 프론트엔드 UI 파일

Admin 시스템의 **프론트엔드**는 별도의 프레임워크 없이 **Vanilla JS 기반 단일 HTML 페이지**로 구현되었습니다 ¹⁵.

편집자 대시보드인 `index.html` 하나로 구성되며, JavaScript를 통해 백엔드 API와 통신하여 기능을 수행합니다

¹⁶. UI에서는 전체 기사 현황(KPI) 표시, 기사 승인/코멘트 입력, 및 각종 작업 버튼 (요약, 보고서 생성, Export 등)을 제공하여 **버튼 클릭만으로 백엔드 작업을 트리거**합니다 ¹⁶.

파일명	경로	주요 기능
index.html	/index.html	QualiJournal 관리자 대시보드의 메인 페이지입니다. 하나의 HTML 파일 내에 뷰어와 제어 패널이 모두 포함되어 있으며, <code><script></code> 섹션을 통해 프론트엔드 로직을 구현합니다. 주요 UI 요소로 전체 기사 수, 승인된 기사 수 등의 KPI 지표를 표시하고, "일일 보고서 생성", "전체 요약 실행", "선정보 요약", "Export MD", "Export CSV" 등 작업 버튼들을 제공합니다 ¹⁶. 각 버튼에는 <code>onclick</code> 이벤트로 대응하는 JavaScript 함수가 연결되어, 누를 때마다 해당 백엔드 API endpoint (예: <code>/api/report</code>, <code>/api/enrich/selection</code>, <code>/api/export/md</code> 등)을 호출하고 결과를 받아 화면에 반영합니다 ¹⁷ ¹⁵. 또한 ADMIN_TOKEN을 필요로 하는 요청의 경우, 초기 설정된 토큰을 <code>Authorization</code> 헤더나 <code>X-Admin-Token</code> 헤더로 붙여 호출하며, 성공/실패에 따라 사용자에게 알림(예: "보고서 생성 완료" 또는 오류 메시지)을 표시합니다.

파일명	경로	주요 기능
app.js	/assets/app.js	프론트엔드 로직의 JavaScript 소스 파일입니다. index.html에서 포함되어 각종 함수와 이벤트 핸들러를 정의합니다. 예를 들어 <code>genReport()</code> 함수 (보고서 생성 API 호출), <code>runEnrichSelection()</code> 함수 (요약 API 호출), 기사 리스트 표시 및 체크박스 제어, API 응답 처리 등을 구현합니다. UI 상의 버튼 클릭 시 이 파일의 함수들이 실행되어 fetch API를 통해 백엔드와 통신하며, 응답에 따라 DOM을 업데이트합니다. (Vanilla JS로 작성되어 특별한 라이브러리 없이 동작)
style.css	/assets/style.css	관리자 UI의 스타일시트 파일로, 대시보드 화면의 레이아웃과 디자인을 정의합니다. 예를 들어 표 및 버튼 스타일, 반응형 레이아웃, 테마 색상 등을 지정하여 사용자에게 보기 좋은 인터페이스를 제공합니다. (라이트/다크 테마 대응, 모바일 화면 대응 등을 포함)
index.html (배포본)	/dist/index.html	정적 빌드된 index 페이지로, 위 <code>assets</code>의 JS/CSS를 번들링/압축한 결과물을 참조하는 HTML입니다. (개발용 <code>index.html</code>과 내용은 유사하나, 해시가 붙은 파일명을 가리킴) Cloud Run 배포 시 이 파일이 사용될 수 있었으나, 현재 Dockerfile 설정상 최신 <code>admin/index.html</code>로 대체되므로 실사용되지는 않습니다.
app.a80cc71a81.js (빌드)	/dist/assets/app.a80cc71a81.js	번들링/압축된 JS 파일로, 개발용 <code>assets/app.js</code>의 난독화/해시버전입니다. 브라우저 캐시 갱신을 위해 파일명에 고유 해시가 포함되어 있습니다. (UI 기능은 동일)

파일명	경로	주요 기능
style.c4cc22854a.css (빌드)	<code>/dist/assets/ style.c4cc22854a.css</code>	압축된 CSS 파일로 , 개발용 <code>assets/style.css</code> 의 최적화 버전입니다. 마찬가지로 해시가 파일명에 포함되어 캐시 무효화에 이용됩니다.
manifest.json	<code>/dist/manifest.json</code>	웹 애플리케이션 매니페스트 파일 로, PWA 구성을 위한 앱 정보가 들어 있습니다. (예: 앱 이름, 아이콘, 시작 URL 등) 현재 Admin UI를 설치형 웹앱으로 활용할 필요는 적으나, 향후 확장 대비하여 포함된 것으로 보입니다.
deployed_index.html	<code>/deployed_index.html</code>	배포된 인덱스 HTML 스냅샷 으로, Cloud Run에 배포된 실제 <code>index.html</code> 내용을 한 때 저장해둔 참고용 파일입니다. (예: 버전 문자열이나 CDN 경로 확인용) 운영 중 배포 버전과 로컬 버전의 비교 또는 문제 분석에 사용되었습니다.

배포/운영 관련 파일

이 섹션의 파일들은 Admin 시스템을 **컨테이너 배포**하거나 **클라우드 서비스와 연동**하기 위한 설정 및 문서들입니다.

파일명	경로	주요 기능
Dockerfile	<code>/Dockerfile</code>	Docker 컨테이너 빌드 스크립트 파일 입니다. Admin 앱을 Cloud Run 등에 배포하기 위한 컨테이너 이미지를 구성합니다. Python 3.11 슬림 이미지를 기반으로, 필요한 패키지를 설치하고 소스코드를 <code>/app</code> 경로에 복사한 뒤 Gunicorn(Uvicorn 워커)을 이용해 <code>server_quali:app</code> 을 8080포트로 구동하는 설정을 합니다. 또한 빌드과정에서 오래된 <code>dist/</code> 폴더를 제거하고 최신 <code>admin/index.html</code> 을 루트로 복사하여 항상 최신 UI가 제공 되도록 처리합니다. 이 Dockerfile을 통해 CI/CD 파이프라인에서 자동으로 이미지를 빌드/배포하게 됩니다.

파일명	경로	주요 기능
.dockerignore	<code>/.dockerignore</code>	Docker 빌드 시 제외/포함 규칙 설정 파일입니다. 불필요한 파일(.git, 캐시, 로그, <code>archive/</code> 산출물 등)을 이미지에 포함하지 않도록 하며, 반대로 필요한 파일(<code>admin/**</code> , 모든 <code>.py/html/js/css</code> , <code>requirements</code> 파일 등)은 확실히 포함되게 예외 규칙을 지정합니다 【57†】 . 특히 구버전 <code>dist/</code> 산출물은 절대 포함되지 않도록* 마지막에 명시하여 항상 최신 프론트엔드가 배포되도록 보장합니다.
.gcloudignore	<code>/.gcloudignore</code>	Cloud Build 소스 배포용 ignore 설정 파일입니다. Dockerfile과 유사하게, GCP <code>gcloud deploy --source</code> 로 배포할 때 업로드 제외/포함할 파일들을 지정합니다 【56†】 . <code>.dockerignore</code> 와 거의 동일한 규칙으로 작성되어, 시크릿/로그 제외 및 최신 소스 포함 을 관리합니다.
Procfile	<code>/Procfile</code>	애플리케이션 실행 프로파일 (Procfile) 로, 플랫폼(예: Heroku)에서 웹 프로세스를 어떻게 실행할지 명시합니다. 내용은 <code>web: python -m uvicorn server_quali:app --host 0.0.0.0 --port 8080</code> 형태이며, 로컬 개발이나 특정 PaaS에서 간단히 Uvicorn 서버를 실행하기 위한 용도로 제공됩니다. (Cloud Run 배포 시는 사용되지 않음)
requirements.txt	<code>/requirements.txt</code>	필수 Python 패키지 목록입니다. FastAPI, uvicorn, gunicorn, python-dotenv 등을 비롯해 QualiJournal 서버 구동에 필요한 라이브러리들의 버전이 지정되어 있습니다. Docker 빌드 시 이 목록을 기반으로 <code>pip install</code> 하여 환경을 구성합니다.
requirements-optional.txt	<code>/requirements-optional.txt</code>	선택적 패키지 목록 으로, 개발 및 확장 기능에 필요한 추가 라이브러리를 나열합니다. 예를 들어 DB 연동용 <code>sqlalchemy</code> 나 <code>google-cloud-sqlconnector</code> , 테스트용 <code>pytest</code> 등이 이 파일에 포함됩니다. 필요에 따라 개발 환경이나 CI에서만 설치됩니다.
run_service.txt	<code>/run_service.txt</code>	Cloud Run 서비스 설정 스냅샷(YAML) 입니다. GCP Cloud Run에 배포된 서비스(<code>quali-admin-domap</code>)의 구성을 <code>gcloud run services describe</code> 로 출력한 내용을 저장한 파일입니다. 여기에는 빌드 ID, 이미지 경로, 환경변수 설정(<code>ADMIN_TOKEN</code> 등 secret로 참조), 리비전 이력과 트래픽 설정 등이 기술되어 있어, 실제 운영 중인 Cloud Run 서비스의 상태를 확인할 수 있습니다. (개발자가 배포 설정이 올바른지 검토하기 위한 참고 자료)

파일명	경로	주요 기능
scheduler.txt	/scheduler.txt	Cloud Scheduler 작업 설정 스냅샷입니다. QualiJournal 일일 보고서 자동 생성 스케줄러(GCP Scheduler)의 job 설정을 나타내는 YAML 조각으로, <code>POST /api/report</code> 를 매일 09:00에 호출하도록 구성된 내용을 담고 있습니다. <code>X-Admin-Token: <토큰></code> 헤더 설정과 타임존(Asia/Seoul) 등이 포함되어 있어, 스케줄러가 올바르게 동작하는지 검증하거나 재설정할 때 참고합니다.

1 2 3 4 5 6 7 8 9 10 12 14 15 16 17 1011_3QualiJournal 관리자 시스템 최종 가이드북.pdf

file://file-6Hu18vVrjQweSb5SZz6a6M

11 1003_5A퀄리뉴스를 전면적으로 리팩터링하여 안정적이고 확장 가능한 시스템으로 구축하려면.pdf

file://file-C1X8pMSvAVd6Xs392FEnwq

13 1004_2A고도화report.pdf

file://file-Ko3WmavWUUXUmLMC1s6fso